

FOREX ENGINE PRO

SHADOW OS v2 Migration Program

SPRINT 1

Runtime Awakening

' SPRINT COMPLETE

Date:	2026-07-06
Sprint Name:	Runtime Awakening
Component:	RuntimeDomainManager
Database:	heliumdb – PostgreSQL 16.10 (Railway)
Tests:	107/107 PASS (61 unit + 11 integration + 25 simulation + 10 stress)
Sprint 0 Reg.:	19/19 PASS (unchanged)
Production:	Live bot UNCHANGED – 0 production files modified
Sacred Constr.:	HONORED – 0 rows of knowledge lost

107

Tests
Passed

0

Tests
Failed

19

API
Methods

25

Simulations
Passed

0

Prod Files
Modified

1. Executive Summary

Sprint 1 delivered RuntimeDomainManager — the first production component of SHADOW OS v2 and the foundational layer on which all future Shadow Engines will communicate. RuntimeDomainManager is now the single, authoritative owner of all 10 SHADOW OS v2 runtime domains.

It enforces optimistic locking (compareAndSwap), records a full immutable version history of every mutation, provides snapshot and rollback capabilities, and includes a consistency check system. 107 tests across 4 suites pass. Zero production files were modified.

Sacred Constraint — HONORED

No deployment, restart, or migration step destroyed the accumulated trading knowledge of the system.

2. Objectives

Phase	Objective	Status
1 — Design	MASTER_ARCHITECTURE.md — single source of truth	' COMPLETE
2 — Impl	RuntimeDomainManager — 19 public methods, full API	' COMPLETE
2 — Impl	Migration 002 — runtime_domain_history table	' COMPLETE
3 — Testing	61 unit tests — all public methods	' 61/61 PASS
3 — Testing	11 integration tests — concurrency, round-trips	' 11/11 PASS
4 — Simulation	25 simulation tests — 6 failure scenarios	' 25/25 PASS
3 — Stress	10 stress tests — throughput + concurrency	' 10/10 PASS
5 — Validation	Sprint 0 regression — 0 production files changed	' 19/19 PASS
6 — Docs	MASTER_ARCHITECTURE.md + PDF	' COMPLETE
6 — Docs	SPRINT_1_REPORT.md + PDF	' COMPLETE

3. MASTER_ARCHITECTURE.md

docs/architecture/MASTER_ARCHITECTURE.md is now the single source of truth for SHADOW OS v2. It contains 15 sections covering the complete system architecture.

Section	Content
1	Purpose and Scope (Sacred Constraint formally stated)
2	Core Design Philosophy (6 Invariants)
3	Four-Layer Memory Hierarchy with ASCII diagrams
4	Complete Component Map — 14 components, responsibility table
5	All 10 Runtime Domain definitions with schemas
6	Manager Hierarchy — 5 managers, API contracts
7	Lifecycle Diagrams — startup, Railway restart, power failure
8	Data Flow Diagrams — trade open pre/post-v2, ShadowLab cycle

9	Component Interaction Matrix — who writes to what
10	Complete Database Schema — 11 tables, 22 indexes
11	API Contracts and error conventions
12	Recovery Sequences — 9 phases, corruption recovery
13	Failure Modes and Mitigations — 10 failure types
14	Implementation Status table
15	Sprint Roadmap — Sprints 0–6

4. Implementation

Files Created

File	Lines	Purpose
telemetry/managers/RuntimeDomainManager.js	530+	Core implementation
telemetry/managers/index.js	19	Barrel export
telemetry/migrations/002_runtime_domain_history.sql	52	Schema migration (Sprint 1)
docs/architecture/MASTER_ARCHITECTURE.md	620+	Architecture document
telemetry/tests/unit/RuntimeDomainManager.test.js	370+	61 unit tests
telemetry/tests/integration/rdm_integration.test.js	180+	11 integration tests
telemetry/tests/simulation/rdm_simulation.test.js	340+	25 simulation tests
telemetry/tests/stress/rdm_stress.test.js	220+	10 stress tests

RuntimeDomainManager API (19 public methods)

Lifecycle:	init() shutdown()	
Core CRUD:	createDomain(domain, value, opts)	!' { created, row }
	getDomain(domain)	!' row null
	listDomains()	!' [row, ...]
	updateDomain(domain, value, opts)	!' row
	patchDomain(domain, patch, opts)	!' row
Optimistic lock:	compareAndSwap(domain, expectedVer, val)	!' { swapped, currentVersion, row }
Snapshots:	takeSnapshot(reason, opts)	!' { snapshotId, createdAt, domainCount }
	getSnapshot(id)	!' row null
	listSnapshots(limit)	!' [row, ...]
	restoreFromSnapshot(id, domains, opts)	!' { restored[], snapshotId }
Version history:	getHistory(domain, limit)	!' [history_row, ...]
	rollback(domain, targetVersion, opts)	!' { domain, rolledBackTo, currentVersion }
Audit:	logConsistency(checkId, severity, desc, detail, opts)	
	resolveConsistency(id, resolution, opts)	
	runConsistencyCheck()	!' { checks, domains, issues, severity }
Health:	ping()	!' { ok, latencyMs }
	getStats()	!' { domains, maxVersion, historyRows, pool }

Production Files — Unchanged

File	Status
index.js (FROZEN)	' 0 lines changed
telemetry/server.js	' 0 lines changed
telemetry/shadowm.js	' 0 lines changed
telemetry/shadowlab.js	' 0 lines changed
telemetry/db-adapter.js	' 0 lines changed
telemetry/index.js	' 0 lines changed

5. Database Changes

Migration 002: runtime_domain_history

```
CREATE TABLE IF NOT EXISTS runtime_domain_history (  
  id          BIGSERIAL  PRIMARY KEY,  
  domain      TEXT      NOT NULL,  
  version     BIGINT     NOT NULL,  
  value       JSONB      NOT NULL,  
  changed_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  changed_by  TEXT       NOT NULL DEFAULT 'system',  
  change_op   TEXT       NOT NULL  
              CHECK (change_op IN ('CREATE', 'UPDATE', 'PATCH', 'CAS', 'RESTORE', 'ROLLBACK', 'SNAPSHOT')),  
  snapshot_id BIGINT     REFERENCES system_snapshots(id) ON DELETE SET NULL,  
  notes       TEXT  
);  
CREATE INDEX idx_rdh_domain_ver ON runtime_domain_history (domain, version DESC);  
CREATE INDEX idx_rdh_changed_at ON runtime_domain_history (changed_at DESC);  
CREATE INDEX idx_rdh_snapshot   ON runtime_domain_history (snapshot_id) WHERE snapshot_id IS NOT NULL;
```

Data Integrity (Sacred Constraint)

Table	Before	After	Status
events	29	29+	' No data lost
shadowm_trades	1	1+	' No data lost
shadowm_timeline	0	0+	' No data lost
runtime_domains	10	10	' All 10 domains present

6. Test Results

Full Test Summary

Suite	Tests	Pass	Fail	Duration
Unit	61	61	0	~1.0s
Integration	11	11	0	~0.6s
Simulation	25	25	0	~0.9s
Stress	10	10	0	~2.9s
TOTAL	107	107	0	~5.9s
Sprint 0 Regression	19	19	0	—

Performance Benchmarks (Stress Tests)

Operation	Latency	Throughput
getDomain()	~2ms	—
updateDomain()	4.9ms avg	~200/s
compareAndSwap() — success	6.7ms avg	~149/s
patchDomain()	4.4ms avg	~227/s

takeSnapshot() — 10 domains	21ms	—
getHistory() — 100 entries	2ms	—
ping()	avg 2ms, max 3ms	—
100KB JSONB read	6ms	—
20 concurrent domain updates	252ms total	—

7. Simulation Results

6 Failure Scenarios — All Verified

Scenario	Tests	Key Verification
SIM-1: Normal Startup	5	All tables found, all 10 domains readable, ping < 200ms
SIM-2: Railway Restart	5	State/version/history survive re-init, snapshot restores
SIM-3: Power Failure	4	PG rollback preserves state, no orphan history entries
SIM-4: Runtime Corruption	5	Detected, logged CRITICAL, rolled back, resolved
SIM-5: Version Conflict	3	Exactly 1 CAS winner, loser retries with fresh version
SIM-6: Database Reconnect	3	Auto-reconnect, committed data preserved, history intact

8. Key Finding — CAS Connection Deadlock

A connection pool deadlock was discovered during simulation SIM-3C and fixed before any production code uses RuntimeDomainManager. This is precisely why simulation testing exists.

Root Cause

The original compareAndSwap() called this.getDomain() in the 'else' branch (CAS failure path) to read the current domain state. getDomain() acquires a new pool connection. But the current client connection is still held — the finally { client.release() } block runs after the return statement. With N concurrent CAS operations and pool.max=N, all N connections are held, all N then try to acquire an N+1th connection, and all N block permanently.

Fix

Read the current domain state within the same client connection (while it is already held), then do ROLLBACK, then return. This eliminates the need for a second connection on CAS failure.

Lesson

Connection pool exhaustion deadlocks do not appear in sequential tests — only under concurrent load. Stress and simulation testing with real pool constraints is not optional.

9. Risks & Known Issues

Active Risks

Risk	Likelihood	Mitigation
Production engines write directly to runtime_domains (bypassing RDM)	Low — Sprint 2	Adapters wrap all writes in Sprint 2
history table grows unbounded over months	Low (long-term)	90-day GC policy; Sprint 5 ValidationManager
Pool connection leak in long-running instance	Low	All connections released in finally; monitored via getStats()

Known Issues

ID	Description	Plan
KI-1	server.js, shadowm.js, shadowlab.js still write directly to runtime_domains	Sprint 2 adapters
KI-2	Pre-Sprint-1 snapshots cannot be restored via restoreFromSnapshot()	Documented; listed in listSnapshots()
KI-3	History GC not yet implemented (90-day policy)	Sprint 5 ValidationManager

10. Final Gate Checklist

Gate	Status
' RuntimeDomainManager is production ready	' PASS
' 107 tests pass, 0 fail	' PASS
' 19 Sprint 0 regression tests pass	' PASS
' MASTER_ARCHITECTURE.md complete	' PASS
' Documentation complete	' PASS
' No production behavior changed	' PASS
' Sacred Constraint honored	' PASS
' Checkpoint created	#ó Pending
' Git committed (user pushes from Shell)	#ó Pending

SPRINT 1 STATUS: ' COMPLETE

STOP — Do NOT begin Sprint 2. Wait for Project Review and Product Owner approval.